



[Home](#)
[Interview Questions](#)
[Java Programs](#)
[Pyramid Programs](#)
[Programming Challenges](#)
[Download Notes](#)
[What's new..?](#)

Multi threading Based Interview Questions

1. Why would you use a synchronized block vs. synchronized method?

Synchronized blocks place locks for shorter periods than synchronized methods.

2. What's the difference between the methods sleep() and wait()

The code `sleep(1000);` puts thread to sleep (Or prevent the thread from executing) for exactly one second. The code `wait(1000);` causes a wait of up to one second. A thread could stop waiting earlier if it receives the `notify()` or `notifyAll()` call.

The method `wait()` is defined in the class `Object` and the method `sleep()` is defined in the class `Thread`.

3. There are two classes: A and B. The class B need to inform a class A when some important event has happened. What Java technique would you use to implement it?

If these classes are threads I'd consider `notify()` or `notifyAll()`. For regular classes you can use the `Observer` interface.

4. What is Runnable interface ? Are there any other ways to make a multithreaded java program?

There are two ways to create new threads:

- Define a new class that extends the `Thread` class
- Define a new class that implements the `Runnable` interface, and pass an object of that class to a `Thread`'s constructor.

The advantage of the second approach is that the new class can be a subclass of any class, not just of the `Thread` class.

5. How can I tell what state a thread is in ?

Prior to Java 5, `isAlive()` was commonly used to test a threads state. If `isAlive()` returned false the thread was either new or terminated but there was simply no way to differentiate between the two.

Starting with the release of Java Tiger (Java 5) you can now get what state a thread is in by using the `getState()` method which returns an Enum of `Thread.States`. A thread can only be in one of the following states at a given point in time.

New, Runnable, Blocked, Waiting, Timed_waiting and Terminated

6. What is the difference between notify and notify All methods ?

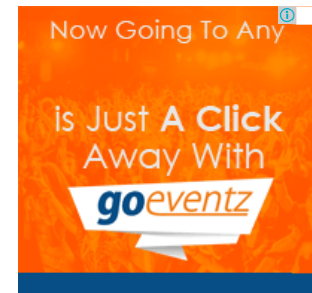
A call to `notify` causes at most one thread waiting on the same object to be notified (i.e., the object that calls `notify` must be the same as the object that called `wait`). A call to `notifyAll`

Like 12k

G+1

2.2k

4 Online Now



Search Program

Follow by Email



Is your savings enough to fulfil your dreams ?



Mutual Fund Investments are subject to market risks, read all scheme related documents carefully.

Labels

Addition of number Advance Java Android AP Series Armstrong
 Number Arrays assertion Binary Addition binary Multiplication
 Binary to Decimal Binary to Hexadecimal C C++ Class Coding
 Standard Collection Conversion Core Java Daemon
 Thread Datatypes Decimal To Binary Decimal to hexadecimal
 Decimal to octal Diamond Durga Sir Durgasoft Encapsulation
 Exception Handling Factorial Features Fibonacci Garbage
 Collection Generic Root GP Series HCF and LCF Hibernate HP
 Series HR Questions BM DB-2 IBPS Identifiers Indore interface
 Interview Interview Question Java

causes all threads waiting on the same object to be notified. If more than one thread is waiting on that object, there is no way to control which of them is notified by a call to `notifyAll`

so, sometimes it is better to use `notify` than `notifyAll`.

7. What is synchronized keyword? In what situations you will Use it?

Synchronization is the act of serializing access to critical sections of code. We will use this keyword when we expect multiple threads to access/modify the same data. It helps prevent dirty read/write and helps keep thread execution clean and separate. For more details on why we need Synchronization and how to use it, you can visit the article on Thread Synchronization as it is a large topic to be covered as an answer to a single question.

8. Why do threads block on I/O?

Threads block on i/o (i.e., Thread enters the waiting state) so that other threads may execute while the i/o Operation is performed. This is done to ensure that one thread does not hold on to resources while it is waiting for some user input - like entering a password.

9. What is synchronization and why is it important?

With respect to multithreading, synchronization is the capability to control the access of multiple threads to shared resources. Without synchronization, it is possible for one thread to modify a shared object while another thread is in the process of using or updating that object's value. This often leads to significant errors. For more details on why we need Synchronization and how to use it, you can visit the article on Thread Synchronization as it is a large topic to be covered as an answer to a single question.

10. Can a lock be acquired on a class?

Yes, a lock can be acquired on a class. This lock is acquired on the class's `Class` object.

11. What's new with the `stop()`, `suspend()` and `resume()` methods in JDK 1.2?

Actually there is nothing new about these methods. The `stop()`, `suspend()` and `resume()` methods have been deprecated as of JDK 1.2.

12. What state does a thread enter when it terminates its processing?

When a thread terminates its processing, it enters the dead state.

13. How do you make threads to wait for one another to complete execution as a group?

We can use the `join()` method to make threads wait for one another

14. What is the difference between yielding and sleeping?

When a task invokes its `yield()` method, it returns to the ready state. When a task invokes its `sleep()` method, it returns to the waiting state.

15. What is the difference between preemptive scheduling and time slicing?

Under preemptive scheduling, the highest priority task executes until it enters the waiting or dead states or a higher priority

Java Program Java4732 JavaEE JavaSE
JDBC JSP Linux Literals Matrix Multi threading Natraj sir
Natraj Technology NCR Notes Number Pattern Number
System Object Octal to binary Octal to Decimal OOPS Oracle
Palindrome Pattern PDF Polymorphism Practice Programs
Prime Number Programming Challenges
Programs Pyramid Real-Life-Example Rectangle
Pattern Recursion Reserved Words Reverse Number SAP SCJP
Serialization Series Series Servlet Session Silver Light
Software Companies Spring Star Pattern String Struts-2 Sum of
digits Swapping Two Array Top 10 triangle tutorial var-args
Method Video Tutorial Web Logic Web Service Written
Test

task comes into existence. Under time slicing, a task executes for a predefined slice of time and then reenters the pool of ready tasks. The scheduler then determines which task should execute next, based on priority and many other factors. You can refer to articles on Operating Systems and processor scheduling for more details on the same.

16. When a thread blocks on I/O, what state does it enter?

A thread enters the waiting state when it blocks on I/O.

17. What is a task's priority and how is it used in scheduling?

A task's priority is an integer value that identifies the relative order in which it should be executed with respect to other tasks. The scheduler attempts to schedule higher priority tasks before lower priority tasks.

18. When a thread is created and started, what is its initial state?

A thread is in the ready state after it has been created and started.

19. What invokes a thread's run() method?

After a thread is started, via its start() method, the JVM invokes the thread's run() method when the thread needs to be executed.

20. What method is invoked to cause an object to begin executing as a separate thread?

The start() method of the Thread class is invoked to cause an object to begin executing as a separate thread.

21. What is the purpose of the wait(), notify(), and notifyAll() methods?

The wait(), notify(), and notifyAll() methods are used to provide an efficient way for threads to wait for a shared resource. When a thread executes an object's wait() method, it enters the waiting state. It only enters the ready state after another thread invokes the object's notify() or notifyAll() methods.

22. What are the high-level thread states?

The high-level thread states are ready, running, waiting, and dead.

23. What is an object's lock and which object's have locks?

An object's lock is a mechanism that is used by multiple threads to obtain synchronized access to the object. A thread may execute a synchronized method of an object only after it has acquired the object's lock. All objects and classes have locks. A class's lock is acquired on the class's Class object.

24. What happens when a thread cannot acquire a lock on an object?

If a thread attempts to execute a synchronized method or synchronized statement and is unable to acquire an object's lock, it enters the waiting state until the lock becomes available.

25. How does multithreading take place on a computer with a single CPU?

The operating system's task scheduler allocates execution time to multiple tasks. By quickly switching between executing tasks, it creates the impression that tasks execute sequentially.

26. What happens when you invoke a thread's interrupt method

while it is sleeping or waiting?

When a task's interrupt() method is executed, the task enters the ready state. The next time the task enters the running state, an InterruptedException is thrown.

27. How can a dead thread be restarted?

A dead thread cannot be restarted. Once a thread is dead, it stays dead and there is no way to revive it.

28. What are three ways in which a thread can enter the waiting state?

A thread can enter the waiting state by invoking its sleep() method, by blocking on I/O, by unsuccessfully attempting to acquire an object's lock, or by invoking an object's wait() method. It can also enter the waiting state by invoking its (deprecated) suspend() method.

29. What method must be implemented by all threads?

All tasks must implement the run() method, whether they are a subclass of Thread or implement the Runnable interface. Without a run() method, a thread cannot execute.

30. What are synchronized methods and synchronized statements?

Synchronized methods are methods that are used to control access to an object. A thread only executes a synchronized method after it has acquired the lock for the method's object or class. Synchronized statements are similar to synchronized methods. A synchronized statement can only be executed after a thread has acquired the lock for the object or class referenced in the synchronized statement.

A synchronized statement can be inside a regular method and vice versa.

31. What are volatile variables

It indicates that these variables can be modified asynchronously. i.e., there is no need for synchronizing these variables in a multi-threaded environment.

32. Where does java thread support reside

It resides in three distinct places

The java.lang.Thread class (Most of the support resides here)
The java.lang.Object class
The java language and virtual machine

33. What is the difference between Thread and a Process

Threads run inside process and they share data.

One process can have multiple threads, if the process is killed all the threads inside it are killed

34. What happens when you call the start() method of the thread

This registers the thread with a piece of system code called thread scheduler. The scheduler is the entity that determines which thread is actually running. When the start() method is invoked, the thread becomes ready for running and will be executed when the processor allots CPU time to execute it.

35. Does calling start () method of the thread causes it to run

No it just makes the thread eligible to run. The thread still has to wait for the CPU time along with the other threads, then at some time in future, the scheduler will permit the thread to run

36. When the thread gets to execute, what does it execute

It executes all the code that is placed inside the run() method.

37. How many methods are declared in the interface runnable

The runnable method declares only one method : public void run();

38. Which way would you prefer to implement threading - by extending Thread class or implementing Runnable interface

The preferred way will be to use Interface Runnable, because by subclassing the Thread class you have single inheritance i.e you wont be able to extend any other class in Java.

39. What happens when the run() method returns

When the run() method returns, the thread has finished its task and is considered dead. You can't restart a dead thread.

40. What are the different states of the thread

The different states of Threads are:

New: Just created Thraed

Running: The state that all threads want to be

Various waiting states : Waiting, Sleeping, Suspended and Blocked

Ready : Waiting only for the CPU

Dead : Story Over

41. What is Thread priority

Every thread has a priority, the higher priority thread gets preference over the lower priority thread by the thread scheduler

42. What is the range of priority integer that can be set for Threads?

It is from 1 to 10. 10 beings the highest priority and 1 being the lowest

43. What is the default priority of the thread

The default priority is 5. It is also called the Normal Priority.

44. What happens when you call Thread.yield()

It causes the currently executing thread to move to the ready state if the scheduler is willing to run any other thread in place of the yielding thread. Yield is a static method of class Thread

45. What is the advantage of yielding

It allows a time consuming thread to permit other threads to execute

46. What happens when you call Thread.sleep()

It causes the thread to while away time without doing anything and without using the CPU. A call to sleep method requests the currently executing thread to cease executing for a specified amount of time as mentioned in the argument to the sleep method.

47. Does the thread method start executing as soon as the sleep time is over

No, after the specified time is over the thread enters into ready state and will only execute when the scheduler allows it to do so. There is no guarantee that the thread will start running as soon as its sleep time is over.

48. What do you mean by thread blocking

If a method needs to wait an indeterminable amount of time until

some I/O occurrence takes place, then a thread executing that method should graciously step out of the Running state. All java I/O methods behave this way. A thread that has graciously stepped out in this way is said to be blocked.

49. What threading related methods are there in object class

wait(), notify() and notifyAll() are all part of Object class and they have to be called from synchronized code only

50. What is preemptive scheduling

Preemptive scheduling is a scheduling mechanism wherein, the scheduler puts a lower priority thread on hold when a higher priority thread comes into the waiting queue. The arrival of a higher priority thread always preempts the execution of the lower priority threads. The problem with this system is - a low priority thread might remain waiting for ever.

51. What is non-preemptive or Time sliced or round robin scheduling

With time slicing the thread is allowed to execute for a limited amount of time. It is then moved to ready state, where it must wait along with all the other ready threads. This method ensures that all threads get some CPU time to execute.

52. What are the two ways of synchronizing the code

Synchronizing an entire method by putting the synchronized modifier in the methods declaration. To execute the method, a thread must acquire the lock of the object that owns the method.

Synchronize a subset of a method by surrounding the desired lines of code with curly brackets and inserting the synchronized expression before the opening curly. This allows you to synchronize the block on the lock of any object at all, not necessarily the object that owns the code

53. What happens when the wait() method is called

The following things happen:

The calling thread gives up CPU
The calling thread gives up the lock
The calling thread goes into the monitor's waiting pool

54. What happens when the notify() method is called

One thread gets moved out of monitors waiting pool and into the ready state and The thread that was notified must reacquire the monitors lock before it can proceed execution

55. Using notify () method how you can specify which thread

should be notified You cannot specify which thread is to be notified, hence it is always better to call notifyAll() method

 Like 0  Share  G+1  1

Posted by Umesh Kushwaha

 +1 Recommend this on Google

Labels: Interview Question, Java, Multi threading

1 comment



Add a comment as Vijay Kumar

Top comments



akash kumar · 2 months ago · Shared publicly

by default for every normal class we create objects are they thread safe (or) are they non thread safe object. If they are non thread safe how can we prove that they are non thread safe using programming

1 · Reply

[Newer Post](#)

[Home](#)

[Older Post](#)

Subscribe to: [Post Comments \(Atom\)](#)



Copyright © 2016. Simple template. Template images by [PLAINVIEW](#). Powered by [Blogger](#).